# Final Project Documentation



## Federal Urdu University

Of Arts, Science & Technology, Gulshan-e-Iqbal Campus



**CineMax Karachi: Online Movie Ticket Booking System**

**Federal Urdu University of Arts, Science & Technology**
Gulshan-e-Iqbal Campus, Karachi
Department of Computer Science

FINAL YEAR PROJECT REPORT

# CineMax Karachi: An Online Movie Ticket Booking System

*Project Thesis and Technical Report*

Submitted in partial fulfilment of the requirements for the degree of
**Bachelor of Science in Computer Science**

## PROJECT MEMBERS


| Name | Roll number | Contribution |
|---|---|---|
| Umair | 19252071 | Administration panel and customer account features |
| Abdul Rafay | 19252064 | Database, booking engine and integrations |
| Muhammad Arqam | 19252054 | User interface design |

**Supervisor**
Muhammad Sadiq
Head of Department, Department of Computer Science


October 2026

# Table of Contents

**Declaration** ..... **iv**

**Certificate of Approval** ..... **v**

**Acknowledgements** ..... **vi**

**Abstract** ..... **vii**

**Chapter 1: Introduction** ..... **1**
1.1 Background ..... 1
1.2 Starting point and problem statement ..... 1
1.3 Aims and objectives ..... 2
1.4 Scope and target users ..... 2
1.5 Contributions ..... 3
1.6 Team and responsibilities ..... 3

**Chapter 2: Literature Review** ..... **4**
2.1 Ticketing as a problem domain ..... 4
2.2 Existing solutions ..... 4
2.3 Architecture and technologies ..... 5
2.4 Concurrency control ..... 6
2.5 Web application security ..... 7
2.6 Research gap ..... 7

**Chapter 3: Requirements Analysis** ..... **8**
3.1 Actors ..... 8
3.2 Functional requirements ..... 8
3.3 Non-functional requirements ..... 9
3.4 Use cases ..... 10
3.5 Business rules ..... 11
3.6 Feasibility ..... 11

**Chapter 4: System Design** ..... **12**
4.1 Architecture ..... 12
4.2 Database design ..... 13
4.3 Seat model ..... 15
4.4 Booking workflow ..... 16
4.5 Booking state machine ..... 17
4.6 Interface design ..... 17
4.7 Key design decisions ..... 17

**Chapter 5: Implementation** ..... **19**
5.1 Development method and tools ..... 19
5.2 The helper layer ..... 19
5.3 Catalogue, shows and the seat map ..... 20
5.4 Seat holds ..... 21
5.5 Checkout and payment ..... 22
5.6 Booking finalisation ..... 23
5.7 E-tickets and signed QR codes ..... 24
5.8 Cancellation, refunds and the ticket scanner ..... 26
5.9 Administration panel ..... 27
5.10 TMDB catalogue seeder and deployment ..... 28

**Chapter 6: Testing and Evaluation** ..... **30**
6.1 Strategy and environment ..... 30

6.2 Unit and integration testing ..... 31
6.3 Concurrency testing ..... 32
6.4 Security and ticket-authenticity testing ..... 32
6.5 Performance ..... 33
6.6 Usability ..... 34
6.7 Evaluation against objectives ..... 34

**Chapter 7: Conclusion** ..... **36**

**References** ..... **37**

**Appendix A: Installation and Configuration** ..... **39**

**Appendix B: Requirement-to-Test Traceability** ..... **41**

# Declaration

We, the undersigned, declare that this report and the software it describes are our own work, carried out as our Final Year Project at the Federal Urdu University of Arts, Science and Technology, Karachi. Where we used the work of others, including open-source libraries, templates and the TMDB data service, it is acknowledged in the text and in the References. This work has not been submitted for any other degree or qualification.

| Name | Roll number | Signature |
|---|---|---|
| Umair | 19252071 | |
| Abdul Rafay | 19252064 | |
| Muhammad Arqam | 19252054 | |

# Certificate of Approval

This is to certify that the Final Year Project titled *CineMax Karachi: An Online Movie Ticket Booking System*, submitted by the students named above, has been examined and approved in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science.

| Role | Name | Signature | Date |
|---|---|---|---|
| Supervisor | Muhammad Sadiq | | |
| Internal examiner | | | |
| External examiner | | | |
| Chairperson, Department of Computer Science | Muhammad Sadiq | | |

# Acknowledgements

We thank our supervisor for guidance throughout the project, and the faculty of the Department of Computer Science at FUUAST for the knowledge on which this work is built. We also thank The Movie Database (TMDB) for its free developer API, and the maintainers of the open-source tools this system depends on: PHP, MariaDB, Apache, Bootstrap, Chart.js, SweetAlert2, qrcode.js and html5-qrcode. Finally, we thank our families for their patience and support.

# Abstract

CineMax Karachi is a web-based cinema ticketing system that lets customers in Karachi choose exact seats on a live seat map, pay, and receive a QR-coded e-ticket, while staff run shows, prices, refunds and door check-in from an administration panel. The project re-engineered a minimal academic PHP application, which had no seat model and was open to SQL injection, into a seat-level booking platform on the same low-cost stack: plain PHP, MariaDB 10.4 and server-rendered pages.

The central problem in ticketing is preventing two customers from buying the same seat. We solve it in three layers: five-minute seat holds stored in the database, a `UNIQUE (show_id, seat_label)` key on both held and sold seats so the database itself refuses a duplicate claim, and a single InnoDB transaction for each booking. Tickets carry an HMAC-signed verification link, so any phone can recognise a forged code. Other features include show scheduling with clash detection, simulated card, JazzCash and Easypaisa payment, a time-based refund policy, a camera ticket scanner, a TMDB-fed catalogue and an analytics dashboard.

Evaluation on Docker showed that in 21 rounds of ten simultaneous requests for the same seat, exactly one request won every time. All 22 business-rule unit checks and all 27 ticket-authenticity checks passed, and public pages answered within 112 ms at the 95th percentile with ten concurrent clients. Plain-text password storage and the simulated payment gateway remain the main limitations.

**Keywords:** online ticket booking, seat reservation, concurrency control, database transactions, PHP, MariaDB, QR code, web application security

CHAPTER 1

# Introduction

This chapter explains why CineMax Karachi was built and states its problem, objectives and scope. Chapters 2 to 7 then cover the literature, requirements, design, implementation, evaluation and conclusions.

## 1.1 Background

Karachi's cinemas screen Hollywood, Bollywood and Lollywood films in multiple halls, with multiple show times each day. Buying a ticket has traditionally meant queuing at the box office or using a chain's own website. Box-office sales give no view of seat availability, cause long queues on release weekends and tie up staff who print tickets by hand.

Online sales solve these problems only when they rest on a reliable model of shows, individual seats and payments. A system that merely records "three tickets for the evening show" cannot show which seats remain, cannot stop two customers from receiving the same seat and cannot be checked at the door. Widely taught, low-cost technologies (PHP, a MySQL-compatible database and small JavaScript libraries) already provide transactions, integrity constraints and interactive pages. This project tests whether they are sufficient to build a ticketing system whose core guarantee, that no seat is ever sold twice, holds under concurrent load.

## 1.2 Starting point and problem statement

The project began from an existing student-template "Online Movie Booking System" written in plain PHP and MySQL. An audit of that baseline found that it had no shows or seats (a booking was one row holding a seat *count*), no concurrency control, SQL built by string interpolation, passwords and card numbers stored in plain text, admin pages that did not all check for a logged-in administrator, and functional defects such as a price-edit query without a `WHERE` clause. We kept its technology and its original database dump, and evolved the system through numbered, re-runnable migrations.

**Problem statement.** Design and implement a secure, seat-level online cinema booking system on a conventional PHP and MySQL stack that guarantees no seat is sold twice under concurrent access, and that supports the full ticket lifecycle from discovery to check-in and refund.

## 1.3 Aims and objectives

The aim is a working, demonstrable booking platform that a single Karachi cinema operator could adopt. Table 1.1 lists the objectives; Chapter 6 evaluates each one against measured results.

**Table 1.1: Project objectives**

| ID | Objective |
|---|---|
| O1 | Scheduled shows (film, cinema, hall, date, time) with automatic detection of hall clashes |
| O2 | Interactive seat map with three seat classes (Economy A to C, Business D to F, First G to H) and per-film prices |
| O3 | Database-level guarantee that a seat cannot be held or sold twice for the same show, verified under concurrent requests |
| O4 | Checkout with snacks, promo codes and a demonstration payment step for card, JazzCash and Easypaisa, storing only the last four digits |
| O5 | E-ticket with a unique reference and QR code, and a scanner that admits each ticket exactly once |
| O6 | Published refund policy with administrator approval, releasing cancelled seats for resale |
| O7 | Protected admin panel with catalogue, shows, promotions, refunds, activity log, dashboard and reports |
| O8 | Real catalogue data from The Movie Database (TMDB) |
| O9 | No SQL injection or cross-site scripting in new code, CSRF protection on state-changing forms, hardened uploads |
| O10 | Reproducible deployment with a single command |

## 1.4 Scope and target users

**In scope:** a customer website (browse, search, film details, show selection, seat map, checkout, simulated payment, e-ticket, booking history, cancellation, profile, reviews), an admin panel for one operator with any number of cinemas and halls, a command-line TMDB seeder, and a Docker environment with XAMPP as an alternative. **Out of scope:** real payment gateways, email or SMS delivery, native mobile apps, multi-tenant operation by competing chains, non-rectangular halls and dynamic pricing.

The system serves four groups: **customers**, who need real seat availability, familiar payment methods, a phone ticket and a clear refund rule; **door staff**, who verify tickets by camera or reference; **cinema managers**, who schedule shows, set prices, approve refunds and track revenue; and **system administrators**, who deploy, migrate and back up the system.

## 1.5 Contributions

- A seat-level data model (`shows`, `seat_holds`, `booking_seats`) added to a legacy schema through re-runnable migrations, without breaking existing bookings.
- A database-enforced, transaction-based double-booking prevention scheme, validated by a repeatable concurrency test.
- A complete ticket lifecycle (hold, pay, issue, scan, cancel, refund) with a documented state machine, and tickets whose QR codes are signed with HMAC-SHA256.
- A small PHP helper layer that brings prepared statements, output escaping, CSRF tokens and flash messages to a framework-less codebase.
- An idempotent, transactional TMDB catalogue seeder.

## 1.6 Team and responsibilities

| Member | Roll number | Responsibilities |
|---|---|---|
| Abdul Rafay | 19252064 | Database design and migrations, shows, seat map and holds, checkout, payment simulation, e-tickets and QR scanner, cancellations and refunds, dashboard, reports, live search, TMDB integration, Docker environment |
| Umair | 19252071 | Admin management pages (movies, prices, snacks, promo codes, coming soon, users, reviews, messages, activity log), admin authentication, sign-up, profile, favourites, reviews, contact, navigation, site pages, end-to-end test passes |
| Muhammad Arqam | 19252054 | User interface design of the customer website and the admin panel |

CHAPTER 2

# Literature Review

This chapter reviews existing booking systems, the architecture and technologies available, and the theory of concurrency control and web security on which the design rests. It closes with the gap the project addresses.

## 2.1 Ticketing as a problem domain

A cinema ticket sells access to a perishable, capacity-limited resource: a seat for the 18:30 show is worth nothing after 18:30, and each seat can be sold exactly once. Unlike ordinary e-commerce stock, the unit is specific (seat E6), so an oversold seat cannot be fixed by back-ordering. Reservation systems therefore share a known structure (Fowler, 2002; Kleppmann, 2017): discrete inventory attached to a timed event, slow interactive selection, fast but atomic commitment, and sharp contention on release days. The standard response to slow selection followed by fast commitment is a **temporary hold**, which sets seats aside for a fixed period during checkout and returns them to sale if the customer leaves. CineMax Karachi uses a five-minute hold.

## 2.2 Existing solutions

We compared categories of system rather than named products, whose features change between releases and whose internals are not public. **Global aggregators** (for example BookMyShow and Fandango) and **chain websites** run on proprietary cinema-management software that a small venue cannot license or host. **Local marketplaces** support JazzCash and Easypaisa but depend on each cinema exposing its inventory. **Academic PHP projects** are free to run; the one this project inherited recorded a booking as one row with a seat count, concatenated request values into SQL and stored passwords in plain text.

**Table 2.1: Capabilities by category of booking system**

| Capability | Aggregator | Chain website | Local marketplace | Academic PHP project | CineMax Karachi |
|---|---|---|---|---|---|
| Seat-level live map | Yes | Yes | Not guaranteed | No | Yes |
| Database-enforced double-booking prevention | Yes | Yes | Not guaranteed | No | Yes |
| Timed seat hold | Yes | Yes | Not guaranteed | No | Yes, 5 minutes |
| Local wallets (JazzCash, Easypaisa) | Not guaranteed | Not guaranteed | Yes | No | Yes, simulated |
| QR e-ticket and door scanning | Yes | Yes | Not guaranteed | No | Yes, signed |
| Self-service cancellation with refund policy | Not guaranteed | Not guaranteed | Not guaranteed | No | Yes |
| Source available to a small operator | No | No | No | Yes | Yes |

*"Not guaranteed" means the capability depends on the individual platform or cinema and is not common to the category.*

The systems with a correct seat model are proprietary, and the systems a small operator can run lack one. CineMax Karachi targets that empty cell.

## 2.3 Architecture and technologies

Web applications conventionally use three tiers: browser, application server and database (Sommerville, 2016). We chose **server-side rendering**, where each URL returns a complete HTML page, over a single-page application, because it needs no separate API layer or JavaScript build chain and is simpler to secure and host. Small JSON endpoints are added only where interaction requires them: seat holds, seat-status polling, live search, promo validation and OTP issue. In Fowler's (2002) terms, the code uses **Page Controller** (one script per page), **Transaction Script** (procedures such as `finalize_booking()` own one business operation each) and **Template View**, with Post/Redirect/Get on every form.

**Table 2.2: Technology stack**

| Layer | Technology | Reason for choice |
|---|---|---|
| Language | PHP 7.4 (also tested on 8.2) | Baseline language; runs on cheap shared hosting; `mysqli` prepared statements, `random_bytes()`, `hash_equals()` |
| Database | MariaDB 10.4 with InnoDB | ACID transactions, row locks, unique and foreign-key constraints, re-runnable `IF NOT EXISTS` migrations |
| Web server | Apache with `mod_php` | Official PHP image; `.htaccess` blocks script execution in upload folders |
| Front end | Bootstrap 5 and 4, jQuery, Chart.js 2.7, SweetAlert2, qrcode.js, html5-qrcode | Responsive templates, dashboard charts, in-browser QR rendering and camera scanning |
| Environment | Docker Compose (web and db services) | One command reproduces the system on any machine |
| Catalogue data | TMDB REST API | Free current and upcoming films with posters, trailers and metadata |

We kept plain PHP rather than migrating to a framework such as Laravel, because a migration would have rewritten every existing page without delivering any missing booking feature, and would have raised hosting requirements.

## 2.4 Concurrency control

The naive booking algorithm is check-then-act: *if seat E6 is free, record E6 as sold*. If two requests read before either writes, both sell the seat, a lost-update anomaly (Berenson et al., 1995). Wrapping the sequence in a transaction does not help on its own, because a plain read under InnoDB's default REPEATABLE READ isolation takes no lock (Silberschatz, Korth and Sudarshan, 2019). Three standard remedies exist (Elmasri and Navathe, 2016; Kleppmann, 2017):

1. **Pessimistic locking:** lock the row before checking it, for example with `SELECT ... FOR UPDATE`.
2. **Optimistic control:** detect conflicts at commit time with a version column and retry.
3. **Constraint-based exclusion:** let a `UNIQUE` key reject the second insert of the same (show, seat) pair, collapsing check and act into one atomic operation.

The third is the most robust for seats, because the invariant "at most one sale per seat per show" maps exactly onto a unique key and stays correct even if a future code path forgets to lock. CineMax Karachi uses it as the primary guarantee, with `FOR UPDATE` locks as a secondary

measure inside the payment transaction. Holds are **leases** (Gray and Cheriton, 1989): each carries an expiry time, and expired holds are treated as free, so an abandoned checkout needs no clean-up protocol.

## 2.5 Web application security

The OWASP Top 10 (OWASP Foundation, 2021) names the risks most relevant here: broken access control (opening another customer's ticket or an unguarded admin page), cryptographic failures (plain-text passwords, stored card numbers), injection, insecure design (double booking, refund abuse) and misconfiguration (executable upload folders). The standard controls are prepared statements against SQL injection (Halfond, Viegas and Orso, 2006), contextual output encoding against cross-site scripting, synchroniser tokens against cross-site request forgery (Barth, Jackson and Mitchell, 2008), and extension allow-lists, content checks and non-executable upload folders against malicious uploads.

A QR code (ISO/IEC 18004) should carry an opaque identifier rather than personal data, and validity must be decided by the server at scan time so that a copied code is admitted once. Signing the identifier with a key held only by the server, using a message authentication code such as HMAC, also lets anyone recognise a code the cinema never issued. For usability, Nielsen's (1994) heuristics motivated the visible hold countdown, disabled sold seats and plain-language error messages.

## 2.6 Research gap

Commercial platforms prevent double booking but are proprietary and costly; affordable academic systems omit the seat model and are insecure. The literature supplies the techniques to close the gap: constraints and transactions for correctness, leases for abandoned checkouts, prepared statements and synchroniser tokens for security, and server-validated, signed identifiers for admission. CineMax Karachi applies them to a framework-less PHP and MariaDB stack and measures whether the guarantee holds under concurrent load.

CHAPTER 3

# Requirements Analysis

Requirements were elicited from an audit of the baseline system, observation of how Karachi cinemas sell tickets, and the platforms reviewed in Chapter 2. They are numbered for traceability to the tests in Chapter 6 and Appendix B, and prioritised on the MoSCoW scale.

## 3.1 Actors

| Actor | Type | Role |
|---|---|---|
| Guest | Primary | Browses films, searches, reads reviews and views show times |
| Customer | Primary | Registered user who books, pays, views tickets, cancels and reviews |
| Administrator | Primary | Cinema staff managing catalogue, shows, prices, refunds and reports; door staff use only the scanner |
| TMDB | Secondary system | Film-data service consulted by the catalogue seeder |

## 3.2 Functional requirements

The system has 50 functional requirements, FR-01 to FR-50. Table 3.1 groups them by area, states the key requirements in each group and counts their priorities.

**Table 3.1: Functional requirements by area**

| IDs | Area | Key requirements | Priority |
|---|---|---|---|
| FR-01 to FR-07 | Catalogue and discovery | List now-showing films by industry; filter by genre, language and cinema; live search after two characters; film page with synopsis, rating, trailer and shows | 3 Must, 2 Should, 2 Could |
| FR-08 to FR-12 | Accounts | Register with a unique email; sign in and return to the requested page; blocked users are signed out; profile and favourites | 3 Must, 1 Should, 1 Could |

| IDs | Area | Key requirements | Priority |
|---|---|---|---|
| FR-13 to FR-19 | Shows and seats | Shows for the next seven days with seats left; seat map coloured by class and state, refreshed every 10 s; at most 10 seats; 5-minute hold that fails as a whole and names conflicting seats | 7 Must |
| FR-20 to FR-26 | Checkout and payment | Hold countdown, snacks, one promo code; total = tickets + snacks − floored discount; card, JazzCash and Easypaisa with OTP; store last four digits only; booking created atomically or not at all | 6 Must, 1 Should |
| FR-27 to FR-33 | Tickets and cancellation | Reference `CNX-YYYY-NNNNN`; e-ticket with a signed QR verification link; Upcoming, Past and Cancelled tabs; customers see only their own tickets; cancellation shows the 100% or 50% refund quote and releases seats | 6 Must, 1 Should |
| FR-34 to FR-36 | Reviews and contact | One rating and comment per film; average counts visible reviews only; contact form | 2 Should, 1 Could |
| FR-37 to FR-50 | Administration | Every admin page gated; catalogue, prices, shows with clash detection, snacks and promotions; booking search; refund approval with reasons; ticket scanning (VALID, ALREADY_USED, CANCELLED, WRONG_DAY, FORGED, NOT_FOUND); dashboard, CSV reports, activity log; atomic TMDB reseed | 7 Must, 6 Should, 1 Could |

## 3.3 Non-functional requirements

### Table 3.2: Non-functional requirements (ISO/IEC 25010)

| ID | Quality | Requirement | Measure |
|---|---|---|---|
| NFR-01 | Correctness | No seat sold twice for one show under any interleaving of requests | Zero duplicate (show, seat) rows; concurrency test |
| NFR-02 | Fault tolerance | A failure during booking or seeding leaves the database unchanged | All multi-row writes in one transaction |
| NFR-03 | Performance | Public pages under 200 ms at the 95th percentile with 10 concurrent users | ApacheBench |
| NFR-04 | Confidentiality | No full card or wallet number stored | Schema and code review |
| NFR-05 | Integrity | Prepared statements for request data; all dynamic output HTML-escaped | Code review, injection tests |
| NFR-06 | Integrity | State-changing actions require a valid CSRF token | Token tests |

| ID | Quality | Requirement | Measure |
|---|---|---|---|
| NFR-07 | Usability | Film page to e-ticket in at most five screens | Flow inspection |
| NFR-08 | Portability | A new developer runs the system with one command | `docker compose up -d --build` |

## 3.4 Use cases

The use-case model has 19 use cases: browsing, search, film details, registration and sign-in for guests and customers (UC-01 to UC-05); seat selection, checkout, e-ticket, history, cancellation and reviews for customers (UC-06 to UC-11); catalogue, show scheduling, promotions, refunds, scanning, reports and user moderation for administrators (UC-12 to UC-18); and catalogue seeding for the operator (UC-19). The central use case is described in full below.

**UC-06/07: Book tickets.** *Preconditions:* the show is Active and has not started, and prices are set for the film.

1. The system displays the seat map with current seat states and class prices.
2. The customer selects one to ten free seats and presses Continue.
3. The system validates the seat labels, releases the customer's earlier holds and inserts a five-minute hold for each seat.
4. Checkout shows a countdown; the customer adds snacks and optionally a promo code.
5. The customer chooses a payment method and enters details.
6. In one transaction the system re-reads the customer's unexpired holds with a lock, re-validates the promo code, inserts the booking and its seats and snacks, increments promo usage, adds loyalty points and deletes the holds.
7. The system commits and shows the e-ticket with its QR code.

*Extensions:* 3a. a seat is already held or sold: the whole hold is rolled back and the conflicting seats are named. 4a. the countdown reaches zero: the customer returns to the seat map. 5a. invalid payment details: the form is shown again and the hold continues. 6a. the hold expired or a seat insert violates the unique key: the transaction rolls back and the customer is told why. *Postcondition:* exactly one booking exists with status Booked, and its seats can no longer be selected by anyone.

## 3.5 Business rules

The rules below are recorded in the project's schema contract, which takes precedence over code if the two disagree.

**Table 3.3: Business rules**

| ID | Rule |
|---|---|
| BR-01 | Rows A to C are Economy, D to F Business, G onwards First; a standard hall has 8 rows of 12 seats |
| BR-02 | A hold lasts 5 minutes; expired holds count as free |
| BR-03 | At most 10 seats per booking |
| BR-04 | Total = tickets + snacks − floor((tickets + snacks) × percent / 100), in whole rupees |
| BR-05 | A promo code is valid if active, unexpired and below its usage limit; usage is incremented inside the booking transaction |
| BR-06 | Loyalty points = floor(total / 100), removed on cancellation, never below zero |
| BR-07 | Reference = `CNX-` + year + booking id padded to five digits; the QR code holds a verification link with the reference and its HMAC-SHA256 signature |
| BR-08 | Cancelling at least 24 hours before the show refunds 100%, later 50%; started shows cannot be cancelled |
| BR-09 | Refunds start Pending and are set to Refunded or Rejected (with a reason) by an administrator |
| BR-10 | Only a Booked ticket can be checked in; it becomes Used |
| BR-11 | Bookings keep copies of film, cinema, date and price text so history does not change with the catalogue |

## 3.6 Feasibility

The project is **technically** feasible because every requirement maps onto standard PHP and InnoDB features, and the riskiest one (NFR-01) is met by a unique key. It is **economically** and **operationally** feasible because all software is free, production needs only a low-cost PHP host, and staff use the scanner in any browser.

CHAPTER 4

# System Design

CineMax Karachi is a server-rendered, three-tier web application. Browsers talk HTTP to Apache and PHP, PHP talks to a single MariaDB database, and an offline command-line seeder talks to the TMDB API. This chapter presents the architecture, the data model, the booking workflow and state machine, and the main design decisions.

## 4.1 Architecture

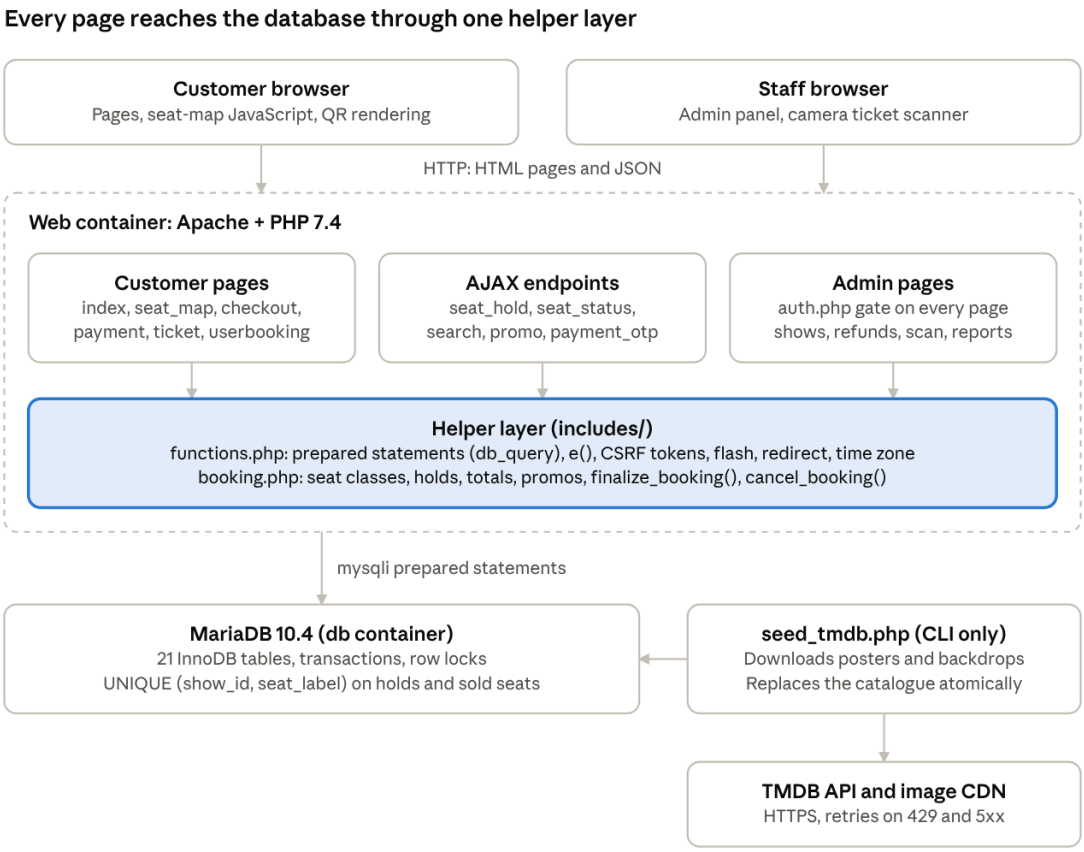



*Figure 4.1: Runtime architecture.* Browsers call page controllers and AJAX endpoints over HTTP; all of them reach MariaDB through the helper layer, and only the command-line seeder talks to TMDB.

Two web applications share one database and one helper layer. The **customer site** consists of page controllers such as `seat_map.php`, `checkout.php`, `payment.php` and `ticket.php`. The **admin panel** (`Admin/`) has one page per feature, and every page starts with `include('auth.php')`, which redirects to the login page unless an administrator is signed in. The **helper layer** (`includes/`) holds all shared logic: `functions.php` for database access, escaping, CSRF tokens and flash messages, and `booking.php` for the booking domain. Five **AJAX endpoints** return JSON for the seat map, live search, promo codes and OTPs.

The code follows a consistent layering without a framework. A controller block at the top of each page reads the request, checks authentication, CSRF token and ownership, calls domain functions, and then either redirects or renders. Because no HTML is output before that block finishes, every form can follow Post/Redirect/Get.

## 4.2 Database design

The database has 21 InnoDB tables: 10 inherited from the baseline dump and 11 added by migration. Figure 4.2 shows the entities and relationships.

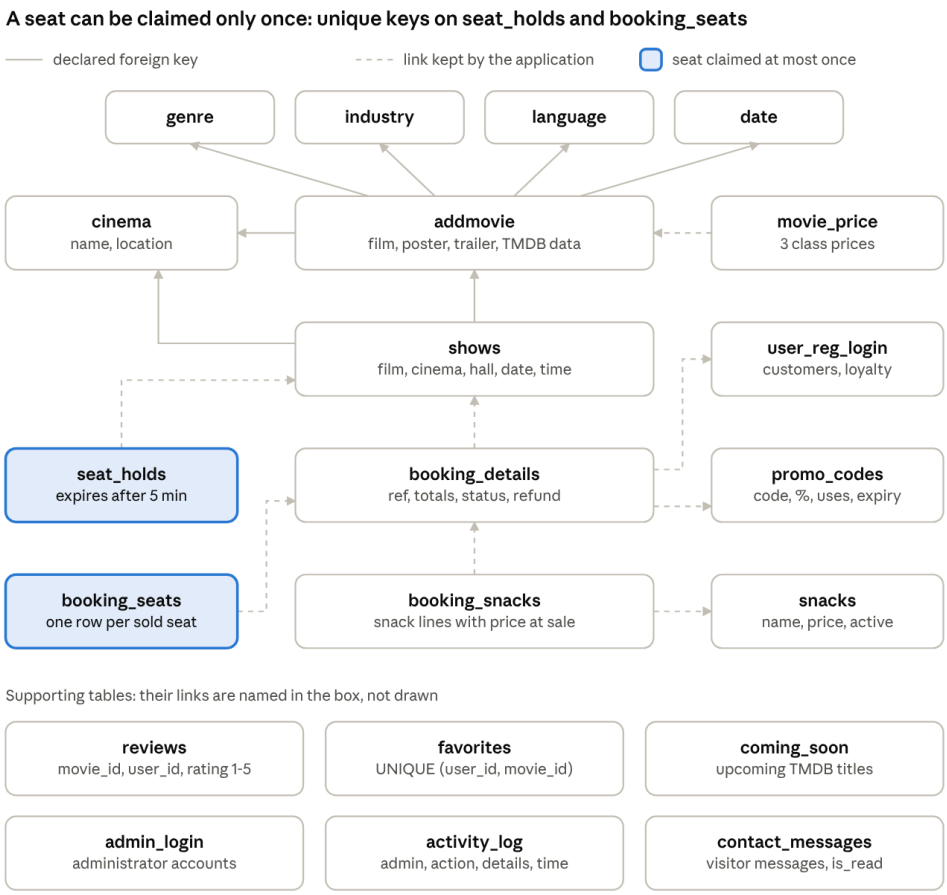



*Figure 4.2: Entity-relationship diagram.* `addmovie` is the hub of the catalogue and `shows` the hub of inventory; the two highlighted tables carry the `UNIQUE (show_id, seat_label)` keys that prevent double booking.

### Table 4.1: Core inventory and transaction tables

| Table | Key columns | Constraints |
|---|---|---|
| `shows` | `show_id`, `movie_id`, `cin_id`, `show_date`, `show_time`, `hall_name`, `total_rows` (8), `seats_per_row` (12), `status` | Foreign keys to film and cinema |
| `seat_holds` | `show_id`, `seat_label`, `user_id`, `expires_at` | **UNIQUE (`show_id`, `seat_label`)** |
| `booking_seats` | `booking_id`, `show_id`, `seat_label`, `seat_class`, `price` | **UNIQUE (`show_id`, `seat_label`)** |
| `booking_details` | Snapshot of film, cinema, date and seats; `booking_ref`, totals, `payment_method`, `status`, `refund_status`, `checked_in_at` | Unique `booking_ref` |
| `promo_codes` | `code`, `discount_percent`, `max_uses`, `used_count`, `valid_until`, `is_active` | Unique `code` |

The two unique keys state the system's main invariant, at most one claim per seat per show, in a form the database enforces on every insert. The new tables are in third normal form, with one deliberate exception: a booking keeps text copies of the film, cinema, date and prices at the time of sale, as an invoice does, so tickets and reports stay accurate after catalogue edits.

The original dump `dbmo.sql` is never edited. Changes are numbered SQL files in `database/migrations/` that use `ADD COLUMN IF NOT EXISTS`, `CREATE TABLE IF NOT EXISTS` and guarded inserts, so any migration can safely be run again. Migration 001 also back-filled totals, references and user links for legacy bookings.

## 4.3 Seat model

A hall is a grid of `total_rows` × `seats_per_row` seats (8 × 12 = 96 by default), labelled `A1` to `H12`. A seat's class depends only on its row: A to C are Economy, D to F Business and G onwards First. A seat's state for a show is derived at read time rather than stored: **sold** if a `booking_seats` row exists, **held** if an unexpired `seat_holds` row exists, and **free** otherwise. With no separate status column, the displayed state cannot drift out of step with the two tables.

## 4.4 Booking workflow

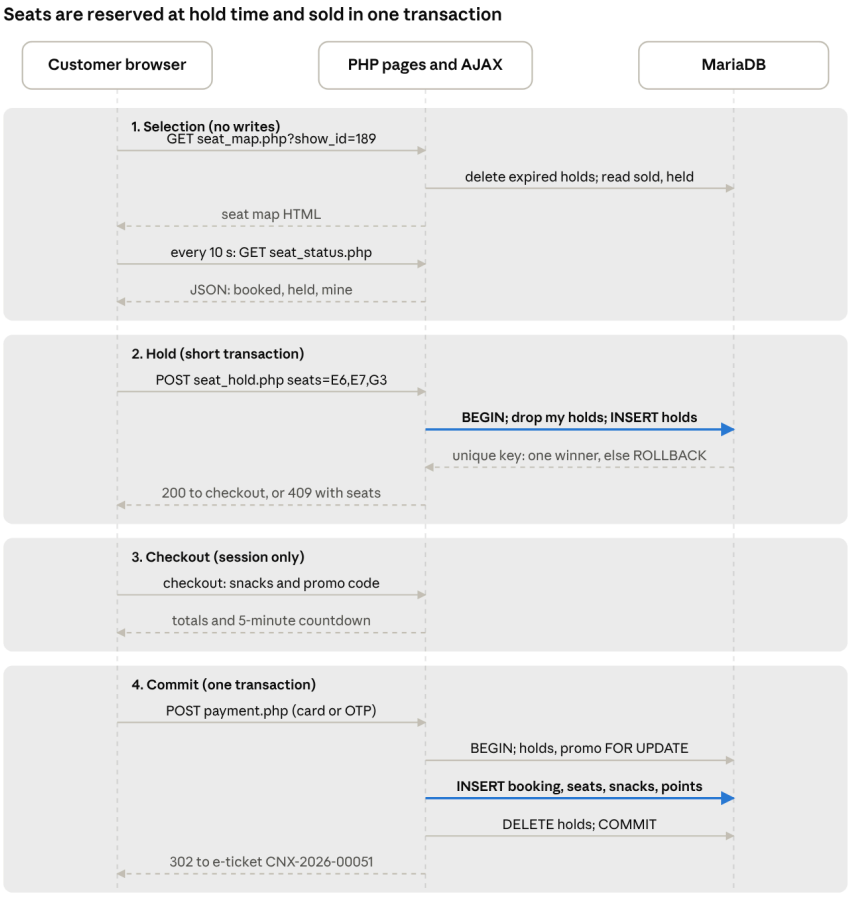



*Figure 4.3: Booking sequence.* Solid arrows are requests, dashed arrows replies; the two highlighted writes are where a unique key can refuse a seat.

The workflow has three phases with different consistency needs:

1. **Selection (no writes).** The browser polls seat status every ten seconds and disables taken seats, but nothing is reserved, so customers who are only looking lock nothing.
2. **Hold (short transaction).** Continue deletes the customer's earlier holds and inserts one hold per seat. Any failed insert, including a unique-key violation caused by a concurrent customer, rolls back the whole hold: the customer gets all requested seats or none.
3. **Commit (one transaction).** On payment, `finalize_booking()` re-reads the customer's unexpired holds `FOR UPDATE`, re-validates the promo code, writes the booking, seats, snacks, promo usage and loyalty points, deletes the holds and commits. The `booking_seats` unique key is the final guard.

## 4.5 Booking state machine

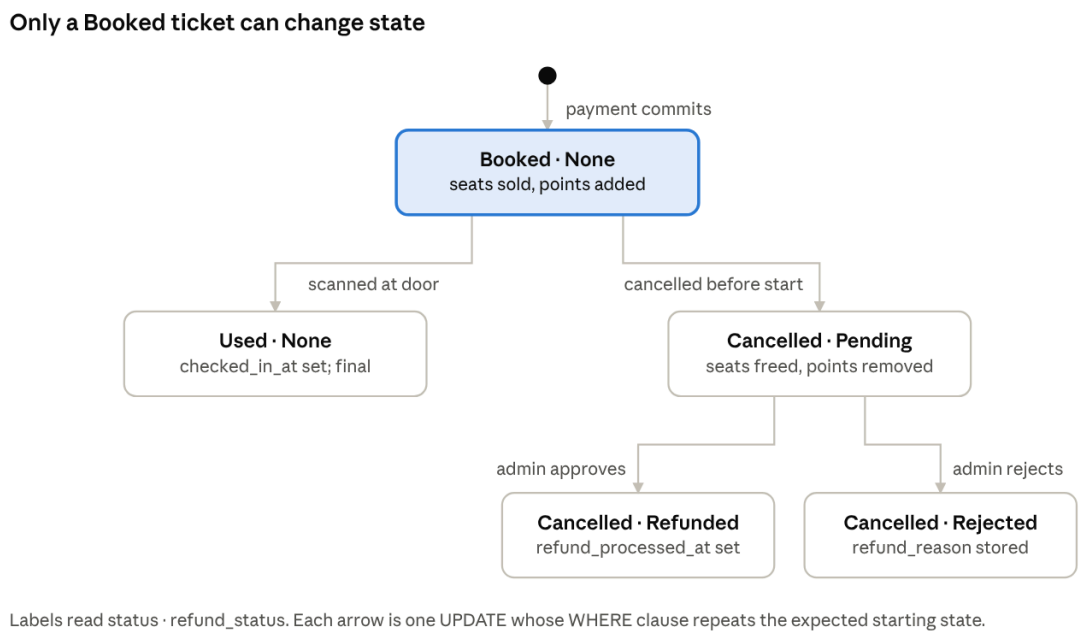



*Figure 4.4: Booking state machine.* A booking leaves the Booked state exactly once, at the door or by cancellation; a cancelled booking then waits for an administrator's refund decision.

Every transition is a conditional `UPDATE` whose `WHERE` clause repeats the expected starting state, for example `WHERE booking_ref = ? AND status = 'Booked'`. The affected-row count tells the code whether the transition happened, so two concurrent attempts, such as a scan and a cancellation of the same ticket, cannot both succeed.

## 4.6 Interface design

Both sites share a light theme of white cards on a light grey page, with the brand's pink-to-coral gradient for actions and status badges that pair a dark label with a pale tint. Seats are `<button>` elements with an `aria-pressed` state, coloured by class, green when selected, orange when held by others and grey when sold. The light theme is applied as override stylesheets, so the purchased dark templates are never edited.

## 4.7 Key design decisions

**Table 4.2: Design decisions**

| Decision | Alternative rejected | Rationale |
|---|---|---|
| Plain PHP with a small helper layer | Migrate to Laravel | Keeps existing pages and shared-hosting deployment; the helper layer supplies the safety features |
| Re-runnable migrations; never edit the dump | Edit or recreate the database | Keeps legacy data and documents history |
| Unique keys for seat exclusivity | Application checks or table locks | Correct under any interleaving and any future code path |
| Holds in a table with lazy expiry | Session holds; cron clean-up | Visible to all customers and enforced by the database; no scheduler to deploy |
| Poll seat status every 10 s | WebSockets | Works on any PHP host; the key, not the display, guarantees correctness |
| QR holds a link with reference and HMAC signature | Plain reference or personal data | No personal data; any phone can confirm the cinema issued it |
| Store only last four payment digits | Store full card numbers | Removes card data from scope |
| Seed catalogue from TMDB by CLI in one transaction | Manual entry or live API calls | Real data without per-request latency; all-or-nothing replacement |

CHAPTER 5

# Implementation

This chapter describes how the system was built: the development method, the helper layer, and the modules that implement the booking lifecycle. Code excerpts are shortened where marked with `...`. Screenshots were taken on 13 October 2026 from the Docker deployment with the TMDB catalogue and demonstration data.

## 5.1 Development method and tools

We used an incremental model: every increment delivered a working, demonstrable slice of the booking flow, so the site never stopped working and defects could be traced to a small change. A foundation step (schema contract, migration 001 and the helper layer) was followed by five booking phases (shows and seat map; checkout and payment; e-ticket and scanner; history, cancellation and refunds; dashboard, reports and search), with administration and account pages built in parallel, then TMDB integration and hardening.

Two practices let the developers change one codebase in parallel. **File ownership** in `TEAM_NOTES.md` assigns every file to one owner; a developer who needs a change in someone else's file writes a numbered request instead of editing it. A **schema contract** (`database/SCHEMA_CONTRACT.md`) defines every table, column and business rule, and wins over code if the two disagree. Work was shared through Git and GitHub, and database changes were delivered only as new migration files. Development used Docker Compose and XAMPP; testing used cURL, Playwright and ApacheBench. The application consists of 9,725 lines of PHP in 86 files, excluding vendored templates and libraries.

## 5.2 The helper layer

Every new page includes `includes/functions.php`, which opens the database connection, starts the session, and sets both PHP and the MySQL session to the Asia/Karachi time zone. Aligning the clocks matters because holds expire by SQL `NOW()` while refund quotes use PHP `time()`; in the UTC Docker container, a show would otherwise be judged five hours wrong.

`db_query()` is the single entry point for parameterised SQL: it prepares the statement, binds typed parameters and returns a result set for reads, the affected-row count for writes, or `false` on failure. Returning the affected-row count is what makes the guarded state transitions of Section 4.5 possible. The codebase makes 220 calls through this wrapper and its `db_one()` and `db_all()` variants. `e()` escapes all output with `htmlspecialchars(..., ENT_QUOTES, 'UTF-8')`, and CSRF tokens are 128 random bits stored in the session and compared with `hash_equals()`:

```
function csrf_token()
{
    if (empty($_SESSION['csrf'])) {
        $_SESSION['csrf'] = bin2hex(random_bytes(16));
    }
    return $_SESSION['csrf'];
}

function csrf_ok()
{
    return isset($_POST['csrf']) && hash_equals(csrf_token(), (string)
$_POST['csrf']);
}
```

## 5.3 Catalogue, shows and the seat map

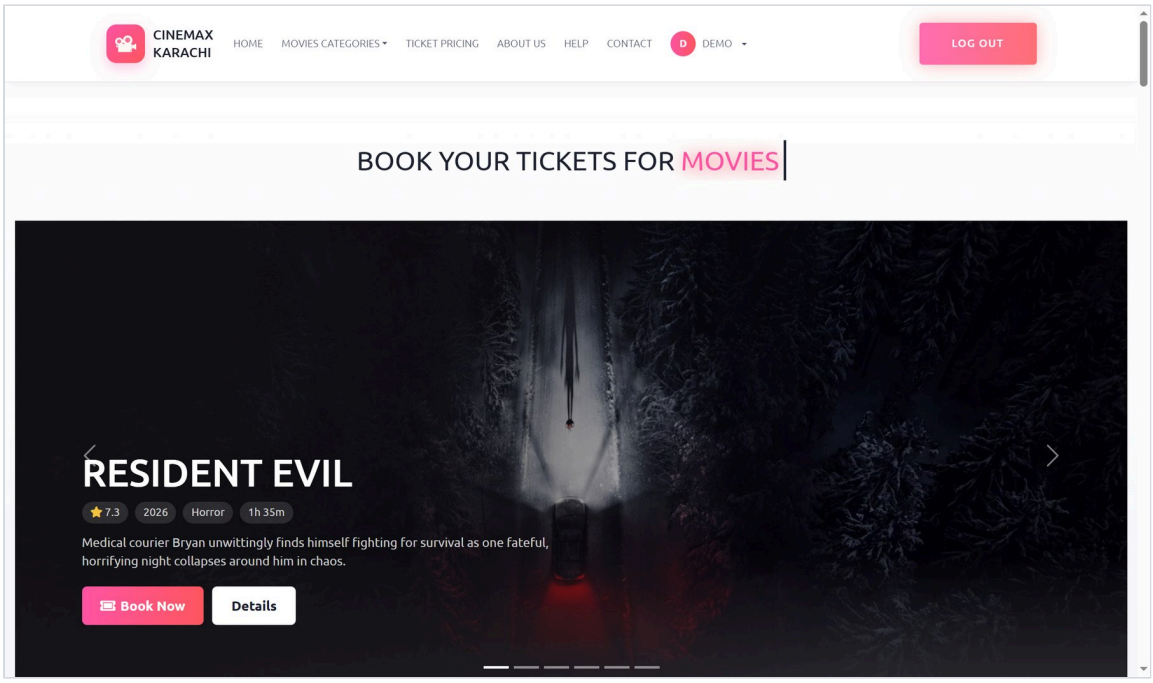



*Figure 5.1: Home page.* The slider shows the most popular current films from TMDB with rating, genre, runtime and a Book Now button.

The movie browser filters by whitelisted genre, language and cinema parameters and offers live search, which escapes SQL `LIKE` wildcards and returns at most six suggestions. `upcoming_shows()` lists a film's active shows for the next seven days with seats left computed in the same query; `TIMESTAMP(show_date, show_time) > NOW()` hides a show the moment it starts, and every booking step re-checks the show with `bookable_show()`.

The seat map renders each seat as a button whose state comes from `seat_states()`, which first deletes expired holds and then returns booked, held-by-others and held-by-me lists. The page polls that state every ten seconds and deselects any seat someone else has taken. Polling is a usability feature, not a correctness mechanism: the server, not the display, refuses a taken seat.

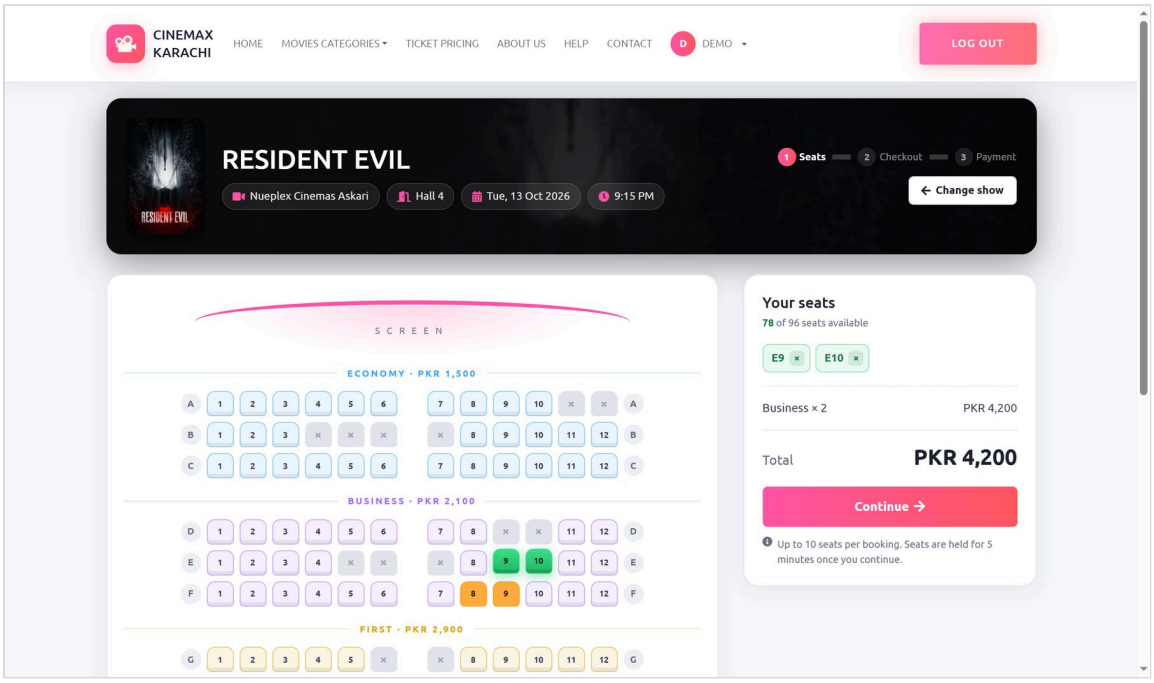



*Figure 5.2: Seat map.* Sold seats are grey with a cross, orange seats are held by another customer, and green seats are this customer's selection; the summary totals the selection by class.

## 5.4 Seat holds

Continue posts the selection to `ajax/seat_hold.php`, which checks sign-in, show status, prices, the 10-seat limit and every seat label, returning a specific HTTP status for each failure (401, 404, 422) and 409 with the conflicting seats. The hold itself is one transaction:

```
function hold_seats($show_id, $user_id, $labels)
{
    release_expired_holds();
    mysqli_begin_transaction(db());
    db_query('DELETE FROM seat_holds WHERE user_id = ?', 'i', [(int) $user_id]);
    $conflicts = [];
```

```
    foreach ($labels as $label) {
        $booked = db_one('SELECT 1 FROM booking_seats WHERE show_id = ? AND
seat_label = ?', ...);
        $inserted = $booked ? false : db_query(
            'INSERT INTO seat_holds (show_id, seat_label, user_id, expires_at)
             VALUES (?, ?, ?, NOW() + INTERVAL 5 MINUTE)', 'isi', [...]);
        if (!$inserted) {
            $conflicts[] = $label;
        }
    }
    if ($conflicts) {
        mysqli_rollback(db());
        return $conflicts;
    }
    mysqli_commit(db());
    return [];
}
```

Three properties matter. Deleting the customer's earlier holds first gives each customer one active selection. When two customers race for the same seat, both may pass the `booking_seats` check, but only one `INSERT` can succeed because of `UNIQUE (show_id, seat_label)`; the loser's seat is reported as a conflict. Any conflict rolls back the whole transaction, so a failed attempt leaves the customer exactly where they were. Expired holds are deleted at the start of every seat read and hold attempt, so a seat frees itself for the next reader without a scheduler.

## 5.5 Checkout and payment

Checkout recomputes every price on the server; the browser's totals are only a preview. Snack quantities are clamped to 0 to 10 and must name active snacks, and the discount is floored so the cinema never gives away more than the advertised percentage. For example, seats E6 and E7 (Business, PKR 1,400 each) and G3 (First, PKR 2,000) plus one soft drink (PKR 250) give PKR 5,050; promo `EID20` takes PKR 1,010, for a total of PKR 4,040. A countdown badge shows the remaining hold time.

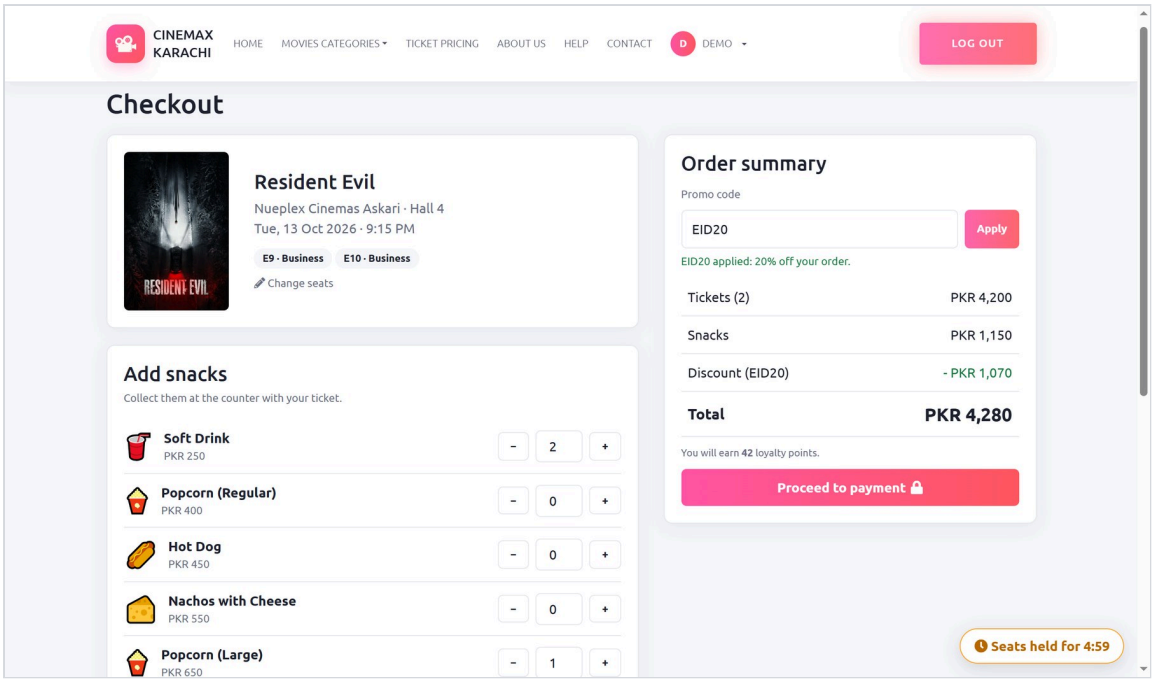



*Figure 5.3: Checkout.* Snacks are added with quantity buttons and promo code EID20 takes 20% off; the badge counts down the hold.

Payment is simulated but validated like a real gateway. For **card**, the server checks a 16-digit number, a future `MM/YY` expiry and a 3 or 4 digit CVV, and keeps only the last four digits. For **JazzCash and Easypaisa**, it checks a Pakistani mobile number (`^03[0-9]{9}$`) and a six-digit one-time password generated with `random_int()`, valid for five minutes and compared with `hash_equals()`. In this demonstration build the OTP is displayed on screen in place of an SMS.

## 5.6 Booking finalisation

`finalize_booking()` is the most important function in the system. It turns a customer's holds into a confirmed booking in a single transaction, and every failure path rolls back:

```
mysqli_begin_transaction($db);
$holds = db_all('SELECT seat_label FROM seat_holds
                 WHERE user_id = ? AND show_id = ? AND expires_at > NOW() FOR
UPDATE', ...);
if (!$holds) {
    return $fail('Your seat hold has expired. Please select your seats again.');
}
if ($promo_code) {
    list($promo, $promo_error) = find_promo($promo_code, true);
    ...
}
$t = checkout_totals($prices, array_column($holds, 'seat_label'), $snack_qty,
$promo);
```

```
$booking_id = ...;
$ref = generate_booking_ref($booking_id);
foreach ($t['seats'] as $seat) {
    $ok = db_query('INSERT INTO booking_seats (booking_id, show_id, seat_label,
seat_class, price)
                    VALUES (?, ?, ?, ?, ?)', 'iissi', [...]);
    if (!$ok) {
        return $fail('Seat ' . $seat['label'] . ' was just booked by someone
else.');
    }
}
...
db_query('DELETE FROM seat_holds WHERE user_id = ?', 'i', [(int) $user['id']]);
mysqli_commit($db);
```

The seats are re-read from the holds inside the transaction, not taken from the form, so a customer cannot add seats by editing the request. Prices are recomputed at commit time. `FOR UPDATE` on the holds and the promo row serialises concurrent attempts, and the `booking_seats` unique key remains the final guarantee. The reference is derived from the auto-increment id in the same transaction, so booking 51 in 2026 becomes `CNX-2026-00051`.

## 5.7 E-tickets and signed QR codes

The ticket page renders only for the booking's owner; any other reference shows "Ticket not found", the same message as for a reference that does not exist. The ticket lists the film, hall, seats, snacks, discount, total and masked payment (for example `Card ****4242`), and prints on its own through a print stylesheet.

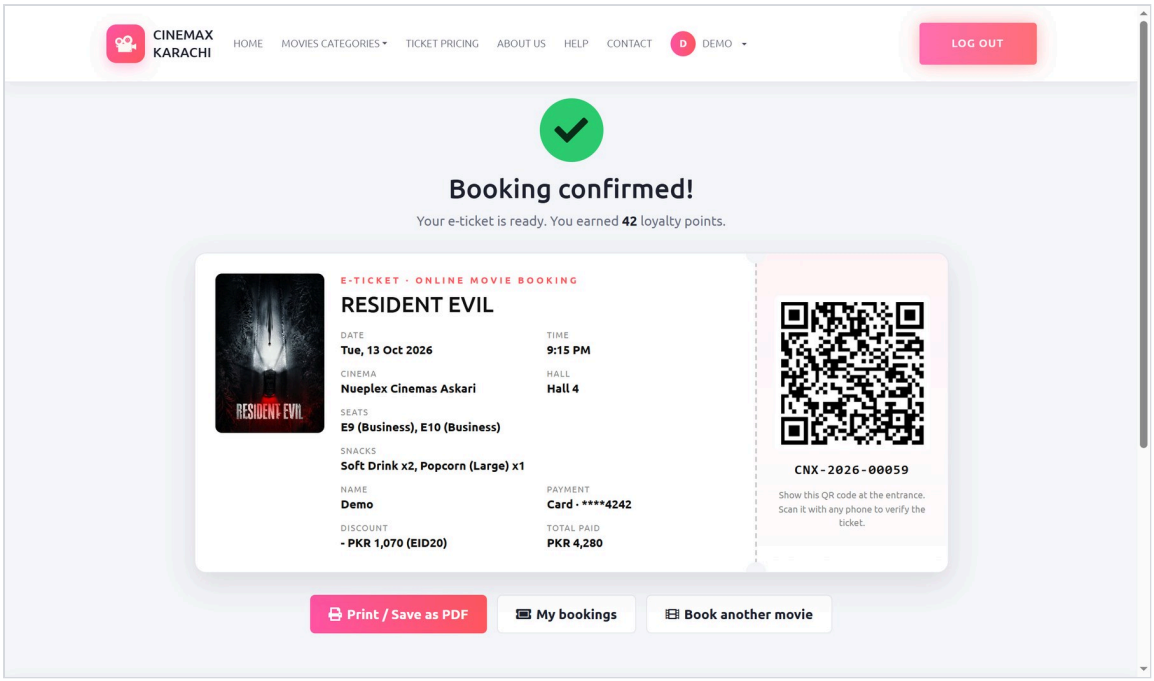



*Figure 5.4: E-ticket.* The ticket lists the show, hall, seats, snacks, discount and payment, and carries the signed QR code.

An early version encoded only the reference in the QR code, so anyone could print a code with a made-up reference. The final version encodes a link to a public verification page carrying the reference and an HMAC-SHA256 signature, computed with a 64-character secret held only on the server and shortened to 24 hexadecimal characters (96 bits):

```
function ticket_signature($ref)
{
    return substr(hash_hmac('sha256', strtoupper(trim((string) $ref)),
ticket_secret()), 0, 24);
}
```

Scanning the code with any phone camera opens `verify.php`, which shows *Genuine ticket*, *Already used*, *Cancelled ticket*, *Show date has passed* or *Not a valid CineMax ticket*. It masks the customer's name and shows no email, phone or price, and it reveals nothing when the signature is wrong, so references cannot be enumerated. Verification is read-only; only the admin scanner admits a ticket.

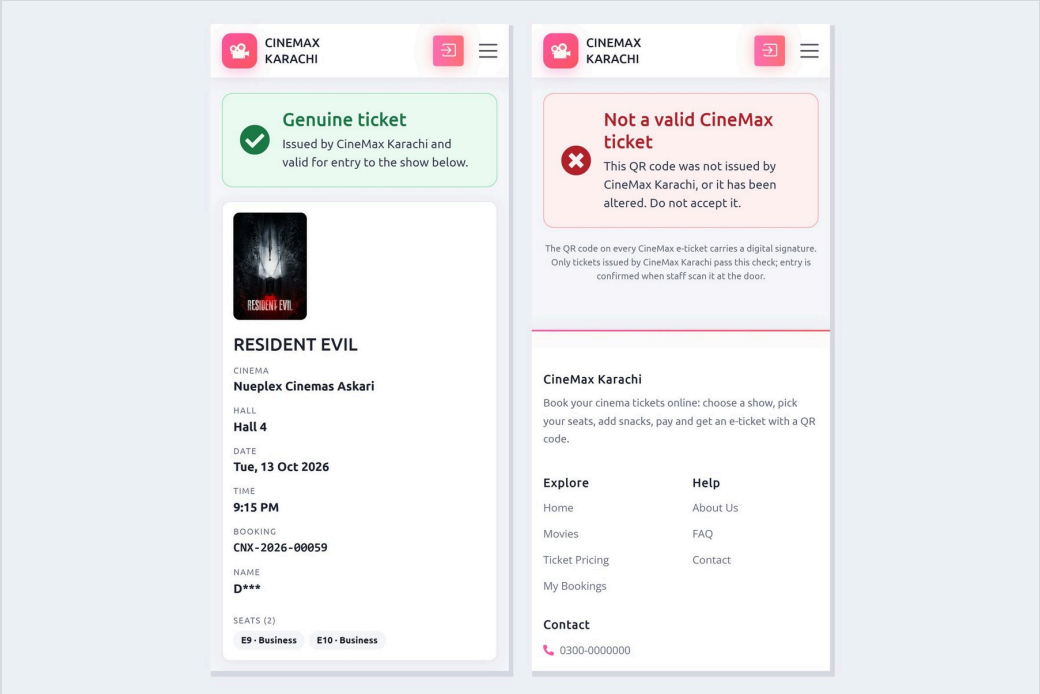



*Figure 5.5: Verifying a ticket with an ordinary phone.* Left: a genuine ticket, with the name masked. Right: the same link with one signature character changed is rejected.

## 5.8 Cancellation, refunds and the ticket scanner

The refund quote is a pure function of the booking and the current time: 100% if the show starts in 24 hours or more, 50% if it starts sooner, and no cancellation once it has started. Cancelling runs one transaction whose `UPDATE` is guarded by `status = 'Booked'`; it then deletes the booking's seat rows, which returns the seats to sale immediately, and deducts the loyalty points. Administrators approve or reject pending refunds with updates guarded by `refund_status = 'Pending'`, so no refund can be processed twice; a rejection records a reason that the customer sees.

The admin scanner reads QR codes from the device camera with html5-qrcode, or accepts a typed reference. The server first checks the signature, then admits the ticket with one atomic `UPDATE ... SET status = 'Used' WHERE booking_ref = ? AND status = 'Booked'`, so two staff scanning the same ticket cannot both get VALID.

**Table 5.1: Scanner outcomes**

| Result | Meaning | Ticket changed? |
|---|---|---|
| VALID | Genuine ticket for a show today; checked in now | Booked to Used |
| ALREADY_USED | Checked in before | No |

| Result | Meaning | Ticket changed? |
|---|---|---|
| CANCELLED | Booking was cancelled | No |
| WRONG_DAY | Genuine, unused ticket for another day | No |
| FORGED | Signature does not match the reference; logged in the activity log | No |
| NOT_FOUND | No booking with this reference | No |

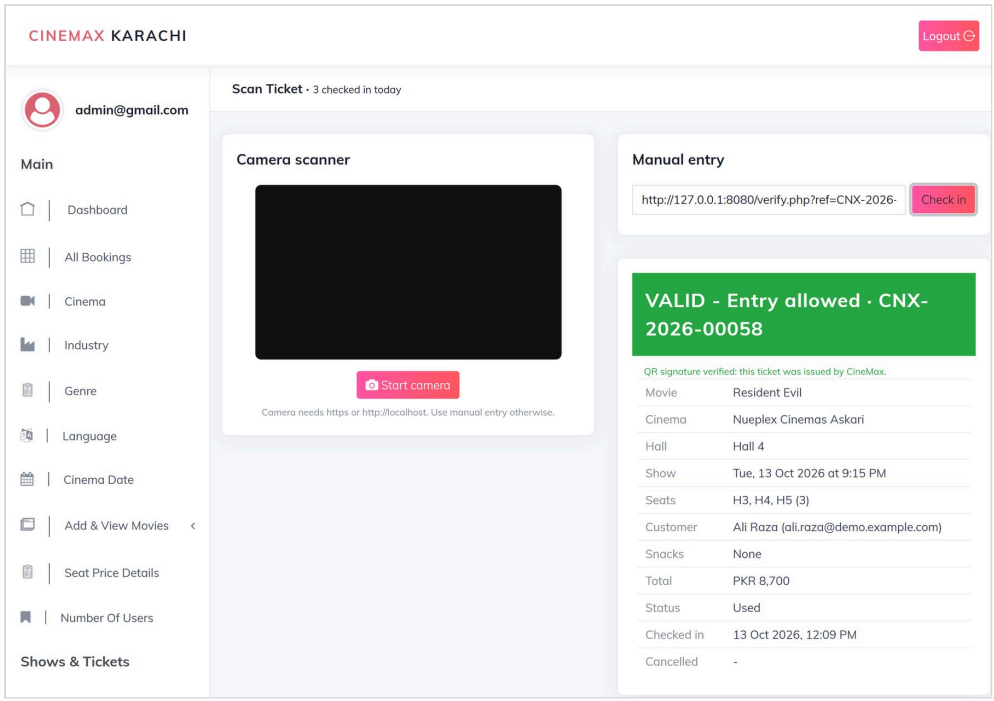



*Figure 5.6: Ticket scanner.* The signed link is verified, the ticket is checked in, and the panel shows the film, hall, time and seats.

## 5.9 Administration panel

Every admin page and delete handler starts with `include('auth.php')`, which redirects and exits unless an administrator is signed in. Catalogue pages share one pattern: an add form, a list with edit and delete, CSRF-protected submissions, flash messages and an activity-log entry for every change. Deleting a cinema or film that still has shows is refused with an explanation. Show scheduling rejects a second show in the same hall at the same date and time, and a show with bookings can only be cancelled, not deleted. Every upload passes through `admin_upload_image()`, which enforces a 2 MB limit and an extension allow-list, confirms the real image type with `getimagesize()`, and saves the file under a random name; the profile-photo folder also denies script execution.

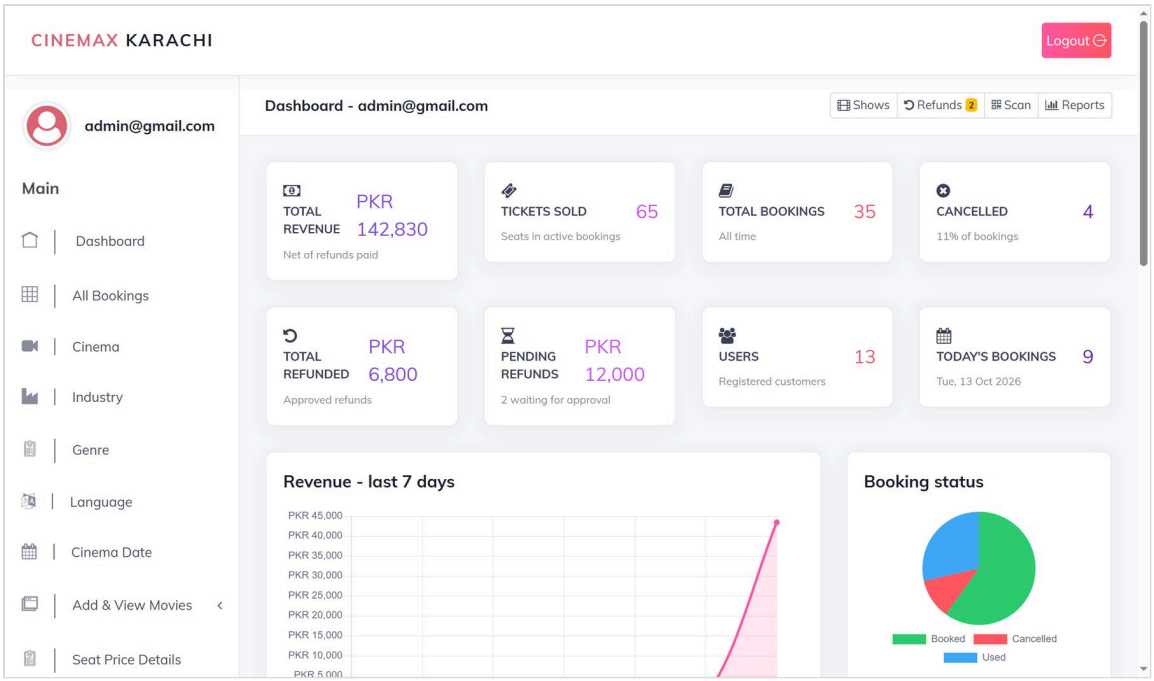



*Figure 5.7: Admin dashboard.* Revenue, tickets, bookings, cancellations, refunds, users and today's bookings, with quick links.

The dashboard computes its tiles in one aggregate query and draws four Chart.js charts (seven-day net revenue, top films, seat-class split and booking status). Reports filter by date range, film and cinema and export CSV; each cell starting with `=`, `+`, `-` or `@` is prefixed with a quote so a hostile customer name cannot run as a spreadsheet formula.

## 5.10 TMDB catalogue seeder and deployment

`database/seed_tmdb.php` runs only from the command line and only with `--yes`. It fetches 16 now-playing English-language films released in the last 120 days, 8 Hindi films from the last 150 days and 6 Pakistani films from the last three years, downloads posters and backdrops, and stops without touching the database if fewer than three films arrive. It then replaces the catalogue in one transaction: films are mapped to industries, priced in three tiers by popularity rank (Economy PKR 1,500 for the top 5, 1,200 for ranks 6 to 15 and 1,000 for the rest), and given four shows (top 10) or three shows over the next seven days. All network work happens before the transaction, so a TMDB outage leaves the database unchanged. Our run produced 30 films (16 Hollywood, 8 Bollywood, 6 Lollywood), 8 coming-soon titles and 112 shows across 9 cinemas.

Deployment is two Docker Compose services: `web` (`php:7.4-apache` with `mysqli`, the repository mounted live) and `db` (MariaDB 10.4, initialised from `dbmo.sql` on first start). `config.php` reads the database host from `DB_HOST`, so the same code runs in Docker and in

XAMPP. `docker compose up -d --build` starts the whole system; Appendix A gives the full steps.

CHAPTER 6

# Testing and Evaluation

All 22 unit checks, all 21 concurrency rounds, all integration, security and end-to-end cases, and all 27 ticket-authenticity checks passed. Public pages answered within 112 ms at the 95th percentile with ten concurrent clients. This chapter describes the strategy, reports each level of testing and evaluates the objectives of Chapter 1.

## 6.1 Strategy and environment

The repository has no automated test suite, so testing combined scripted checks written for this evaluation with the team's manual end-to-end passes. The database was dumped before the scripted tests and restored afterwards, and we confirmed that booking ids, holds and promo counts matched their earlier values.

**Table 6.1: Test levels**

| Level | Scope | Method |
|---|---|---|
| Unit | Seat classes, labels, totals, discounts, refunds, loyalty, references, escaping, CSRF | 22 assertions in the PHP CLI against `includes/booking.php` |
| Integration | AJAX endpoints and pages with a real database and sessions | cURL scripts with one cookie jar per user |
| Concurrency | Double-booking prevention | 10 signed-in users holding the same seat at once |
| Security | Injection, XSS, CSRF, access control, ownership, ticket forgery | cURL scripts and code review |
| Acceptance | Full customer and admin flows in a browser | The team's demonstration scripts on XAMPP with PHP 8.2 |
| Performance | Throughput and latency | ApacheBench, 200 requests per page at concurrency 1 and 10 |

Tests ran on 12 and 13 October 2026 under Docker on Windows 10 (PHP 7.4.33, MariaDB 10.4.34) with 30 films, 112 shows, 26 bookings and 13 users. The main test show was show 189 at 18:30 on 12 October, less than 24 hours ahead, so cancellations quoted 50%.

## 6.2 Unit and integration testing

The 22 unit checks covered every pure business rule. Examples: rows A, D and G map to Economy, Business and First; `H12` is valid and `I1`, `A13` and `1A` are not; booking 12 in 2026 becomes `CNX-2026-00012`; EID20 on PKR 4,400 gives a total of 3,520; a 33% discount on 1,000 is floored to 330; cancelling 48 hours ahead refunds 100% and 1 hour ahead 50%, floored; `e()` escapes all four HTML-special characters; and `csrf_ok()` rejects a wrong token. **All 22 passed.**

The hold endpoint refused a non-existent seat (`Z99`), an eleventh seat, a request without a session (HTTP 401) and an unknown show (HTTP 404), each with its specific message. In the end-to-end booking test E01, a customer held E6, E7 and G3, applied `eid20`, added a soft drink and paid by card. The database then held booking `CNX-2026-00051` with seats "E6, E7, G3", discount PKR 1,010, total PKR 4,040, card digits `4242` only, three `booking_seats` rows at their sale prices and no remaining holds, exactly matching the hand calculation in Section 5.5.

**Table 6.2: Scanning and cancellation tests**

| ID | Scenario | Observed | Result |
|---|---|---|---|
| A04 | Scan without CSRF token | HTTP 403 | Pass |
| A05 | Scan a ticket twice; scan an unknown reference | VALID, then ALREADY_USED; NOT_FOUND | Pass |
| E02 | Book seat B5 (PKR 1,000) and cancel | Cancelled, refund Pending, PKR 500; seat released | Pass |
| S06 | Cancel without CSRF token | No change | Pass |
| A06 | Cancel an already-scanned ticket | No change; status still Used | Pass |

In the team's browser passes on PHP 8.2, every customer and administration flow worked end to end. Those passes also exposed one defect: from PHP 8.1, `mysqli` throws an exception on a duplicate key, so a customer who loses a seat race sees an error page instead of the friendly message. The seat is still not double-sold, because the unique key rejects the insert; the fix is to catch the exception in the data-access wrapper.

## 6.3 Concurrency testing

This test evaluates objective O3 directly. Ten customers, each with their own session, sent `POST ajax/seat_hold.php` for the same seat at the same moment as ten parallel cURL processes. The first run targeted seat H12; a second run repeated it for 20 further seats (C1 to C10 and D1 to D10).

**Table 6.3: Concurrency results**

| Run | Rounds | Requests | Rounds with exactly one winner | Held (HTTP 200) | Refused (HTTP 409) |
|---|---|---|---|---|---|
| C01: seat H12 | 1 | 10 | 1 of 1 | 1 | 9 |
| C02: 20 seats | 20 | 200 | 20 of 20 | 20 | 180 |
| **Total** | **21** | **210** | **21 of 21** | **21** | **189** |

Every refused customer received a message naming the seat, such as "Sorry, E6 was just taken by someone else." The application took no lock before the insert, so the requests genuinely raced and the unique key decided every race. Because the guarantee lives in the database, it does not depend on request timing, the number of web workers or the PHP version.

## 6.4 Security and ticket-authenticity testing

**Table 6.4: Security tests**

| ID | Test | Observed | Result |
|---|---|---|---|
| A01 | Open 8 admin pages, including a delete handler, without logging in | 8 of 8 redirected to the login page | Pass |
| A02 | Run the TMDB seeder over HTTP | HTTP 404 | Pass |
| S01 | Open another customer's ticket by reference | "Ticket not found"; nothing revealed | Pass |
| S03 | `' OR 1=1 --` in customer and admin login | Login refused | Pass |
| S04 | `<script>` in live search | Empty JSON; payload not reflected | Pass |
| S05 | Blocked customer signs in | Refused with explanation | Pass |

After signed QR codes were introduced, 27 further checks ran on 13 October 2026: 10 unit checks of the signing functions and 17 through the running site. A genuine ticket's link showed "Genuine ticket" with the seats and a masked name and no email. An altered or missing signature, or another booking's reference with this signature, showed "Not a valid CineMax ticket" and revealed nothing. At the door, the genuine link checked in as VALID and then ALREADY_USED; a forged code returned FORGED, left that booking unchanged and was logged; an unused ticket for an earlier date returned WRONG_DAY and stayed Booked; and a cancelled booking was refused on both the page and the scanner. **All 27 passed.**

## 6.5 Performance

ApacheBench ran inside the web container to remove network effects, sending 200 requests per page with one client and then with ten concurrent clients. Pages that need a login reused one session cookie.

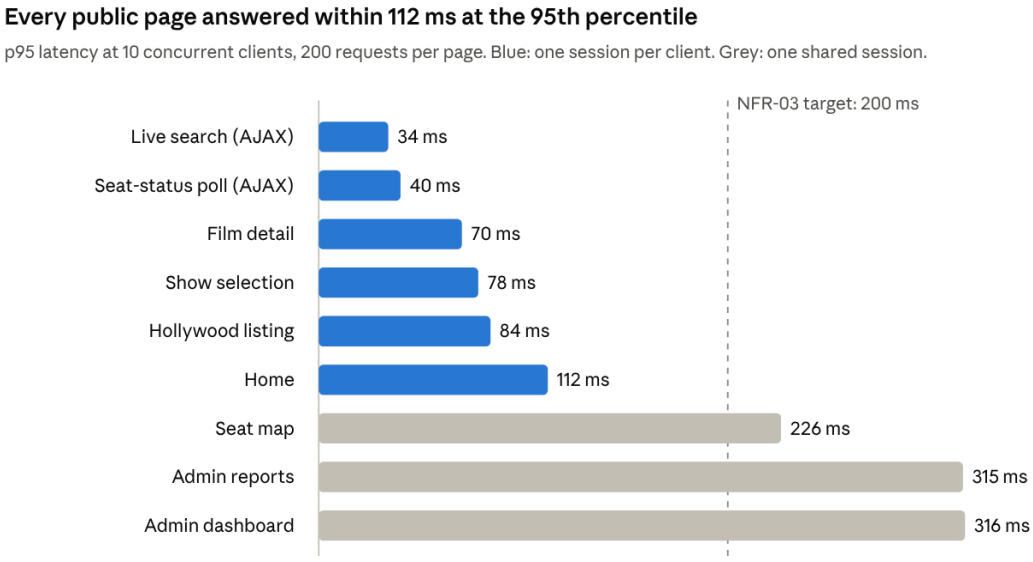



*Figure 6.1: 95th-percentile latency at ten concurrent clients.* All six public pages stay well under the 200 ms target; the three pages over it were tested with one shared session cookie.

Public pages served 130 to 176 requests per second, 4.8 to 5.9 times their single-client throughput, and the seat-status poll served 298 requests per second, enough for 2,980 customers polling every ten seconds on one host (298 × 10). The three slower pages were tested with a single shared session, and PHP's file-based session lock serialises concurrent requests from one session. Real customers each have their own session, so this reflects the test setup, not a capacity

limit; calling `session_write_close()` early on read-only pages would remove it. The data set held 26 bookings and 112 shows, so these figures measure relative cost on one host, not production capacity.

## 6.6 Usability

The booking flow takes five transitions from film page to e-ticket, meeting NFR-07. The seat map shows a live total, checkout shows a hold countdown that turns red in the final minute, taken seats are disabled before a click, and conflicts produce specific messages. Seats are buttons with `aria-pressed` and the countdown is an ARIA live region. Two accessibility gaps remain: held and selected seats are distinguished by colour alone, and seat buttons lack descriptive labels such as "Seat E6, Business, available".

## 6.7 Evaluation against objectives

**Table 6.5: Objectives and evidence**

| Objective | Evidence | Assessment |
|---|---|---|
| O1 Shows with clash detection | Shows page with clash query; 112 shows generated | Met |
| O2 Seat-level booking | Seat map; unit checks; E01 class prices | Met |
| O3 No double booking | Unique keys; 21 of 21 rounds with one winner | Met |
| O4 Checkout and payment | E01: snacks, EID20, card; last four digits only | Met (payment simulated by design) |
| O5 Verifiable tickets | A05; 27 of 27 authenticity checks | Met |
| O6 Cancellation and refunds | Refund unit checks; E02; approve and reject | Met |
| O7 Administration and analytics | A01; dashboard, reports, activity log | Met |
| O8 Real catalogue | 30 TMDB films, 8 coming-soon titles | Met |
| O9 Security baseline | Prepared statements, escaping, CSRF on admin and cancellation, upload hardening; S01 to S05 | Partly met: no password hashing, no CSRF on checkout and payment, no customer session regeneration, no login throttling |
| O10 Reproducible deployment | Every test ran on `docker compose up -d --build` | Met |

Nine of the ten objectives were fully met. O9 was met for injection, XSS, access control and CSRF on administrative and cancellation actions, while password hashing, CSRF on the checkout endpoints, customer session regeneration and login throttling remain outstanding.

CHAPTER 7

# Conclusion

This project set out to build a seat-level online cinema booking system for Karachi, on a conventional PHP and MariaDB stack, that never sells a seat twice. CineMax Karachi delivers the full ticket lifecycle. Customers browse films imported from TMDB, select exact seats on a live map, pay through a simulated card, JazzCash or Easypaisa step, and receive an e-ticket with an HMAC-signed QR code. Staff schedule shows with clash detection, manage prices and promotions, approve or reject refunds, admit customers with a camera scanner and track sales on a dashboard with CSV reports.

The central contribution is the concurrency design: five-minute seat holds stored in the database, a `UNIQUE (show_id, seat_label)` key on both held and sold seats, and one InnoDB transaction per booking. The evaluation confirmed it: in 21 rounds of ten simultaneous requests for one seat, exactly one request won every round. All 22 unit checks and all 27 ticket-authenticity checks passed, and public pages answered within 112 ms at the 95th percentile with ten concurrent clients.

Nine of the ten objectives were fully met. Objective O9 was partly met, and five changes are required before the system takes real payments: password storage with `password_hash()`, CSRF tokens on the checkout and payment requests, session-id regeneration at customer sign-in, login throttling, and integration of a real payment gateway.

The project also shows that an inherited codebase can be improved without a rewrite. The original database dump was never edited, every schema change was a re-runnable migration, a 145-line helper layer supplied prepared statements, output escaping and CSRF protection, and file ownership let the team work in parallel. With the five changes above in place, CineMax Karachi gives a small Karachi cinema reliable online, seat-level booking on low-cost hosting.

# References

Barth, A., Jackson, C. and Mitchell, J.C. (2008) 'Robust defenses for cross-site request forgery', in *Proceedings of the 15th ACM Conference on Computer and Communications Security (CCS '08)*. New York: ACM, pp. 75–88.

Berenson, H., Bernstein, P., Gray, J., Melton, J., O'Neil, E. and O'Neil, P. (1995) 'A critique of ANSI SQL isolation levels', in *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data*. New York: ACM, pp. 1–10.

Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of Database Systems*. 7th edn. Hoboken, NJ: Pearson.

Fowler, M. (2002) *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley.

Gray, C.G. and Cheriton, D.R. (1989) 'Leases: an efficient fault-tolerant mechanism for distributed file cache consistency', in *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP '89)*. New York: ACM, pp. 202–210.

Halfond, W.G.J., Viegas, J. and Orso, A. (2006) 'A classification of SQL-injection attacks and countermeasures', in *Proceedings of the IEEE International Symposium on Secure Software Engineering*. Arlington, VA: IEEE.

ISO/IEC (2011) *ISO/IEC 25010:2011 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. Geneva: International Organization for Standardization.

ISO/IEC (2015) *ISO/IEC 18004:2015 Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification*. Geneva: International Organization for Standardization.

Kleppmann, M. (2017) *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media.

Nielsen, J. (1994) 'Enhancing the explanatory power of usability heuristics', in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '94)*. New York: ACM, pp. 152–158.

OWASP Foundation (2021) *OWASP Top 10:2021*. Available at: https://owasp.org/Top10/ (Accessed: 12 October 2026).

Silberschatz, A., Korth, H.F. and Sudarshan, S. (2019) *Database System Concepts*. 7th edn. New York: McGraw-Hill.

Sommerville, I. (2016) *Software Engineering*. 10th edn. Harlow: Pearson.

The Movie Database (n.d.) *TMDB API Documentation (v3)*. Available at: https://developer.themoviedb.org/docs (Accessed: 12 October 2026).

*Software used:* PHP 7.4 and 8.2, MariaDB 10.4, Apache HTTP Server, Docker Compose, Bootstrap, jQuery, Chart.js, SweetAlert2, Font Awesome, qrcode.js, html5-qrcode, and the FlixGo and Bootstrapious templates. This product uses the TMDB API but is not endorsed or certified by TMDB.

APPENDIX A

# Installation and Configuration

## A.1 Docker (recommended)

Prerequisite: Docker Desktop, or Docker Engine with the Compose plugin. Start the system, then apply the migrations in number order:

```
docker compose up -d --build
for f in database/migrations/*.sql; do docker compose exec -T db mysql -uroot dbmo <
"$f"; done
```

The site is then at `http://127.0.0.1:8080` and the admin panel at `/Admin/login.php`. To load a fresh TMDB catalogue (this replaces films, shows and bookings), back up first:

```
docker compose exec -T db mysqldump -uroot dbmo > backup.sql
docker compose exec web php database/seed_tmdb.php --yes
docker compose exec web php database/seed_demo.php
```

## A.2 XAMPP

Place the project in `C:\xampp\htdocs\fyp_pro`, create the database `dbmo` in phpMyAdmin, import `dbmo.sql` and then each migration in number order, and open `http://localhost/fyp_pro/`.

## A.3 Configuration and demonstration data

| Setting | Location | Default |
|---|---|---|
| Database host | `DB_HOST` environment variable | `localhost` (`db` under Docker) |
| TMDB key | `TMDB_API_KEY` or `includes/secrets.php` | none |
| Ticket signing key | `TICKET_SECRET` or `includes/secrets.php` | none |

| Setting | Location | Default |
|---|---|---|
| Hold duration, seat limit | `HOLD_MINUTES`, `MAX_SEATS` in `includes/booking.php` | 5 minutes, 10 seats |

| Role | Login | Password |
|---|---|---|
| Administrator | admin@gmail.com | 12345 |
| Customer | Demo@outlook.com | 123 |

Demo payment: card 4242 4242 4242 4242 with any future expiry and CVV 123, or JazzCash or Easypaisa with 03001234567 and the OTP shown on screen. Promo codes: EID20 (20%), FYP10 (10%), WELCOME50 (50%, 5 uses).

APPENDIX B

# Requirement-to-Test Traceability

| Requirement | Evidence | Outcome |
|---|---|---|
| FR-14, FR-15 Seat map and classes | Unit checks (6.2); E01 | Pass |
| FR-17 to FR-19 Seat limit, holds and conflicts | Hold-endpoint tests; C01, C02 | Pass |
| FR-21, FR-22 Promo codes and totals | Unit checks; E01 | Pass |
| FR-24 Last four digits only | E01 | Pass |
| FR-25, FR-26 Atomic booking | E01; code review of rollback paths | Pass |
| FR-27, FR-28 Reference and signed QR | Unit checks; 27 authenticity checks (6.4) | Pass |
| FR-31 Own tickets only | S01 | Pass |
| FR-32, FR-33 Cancellation and seat release | Unit checks; E02, S06, A06 | Pass |
| FR-37 Admin authentication | A01 | Pass |
| FR-44 Ticket scanning | A04, A05; authenticity checks | Pass |
| NFR-01 No double sale | C01, C02 (21 of 21 rounds) | Pass |
| NFR-03 Latency | Section 6.5: public pages at most 112 ms | Pass |
| NFR-05 Injection and escaping | S03, S04; unit check of `e()` | Pass |
| NFR-06 CSRF | Unit check; S06, A04 | Partly met: enforced on admin and cancellation, absent on checkout and payment |
| NFR-08 One-command setup | Every test ran on `docker compose up` | Pass |